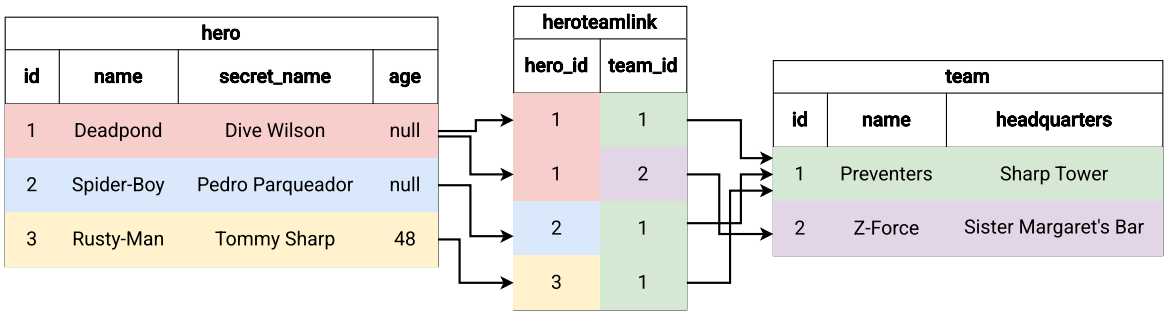
# Create Data with Many-to-Many Relationships

Let's continue from where we left and create some data.

We'll create data for this same **many-to-many** relationship with a link table:




We'll continue from where we left off with the previous code.

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine


class HeroTeamLink(SQLModel, table=True):
    team_id: int | None = Field(default=None, foreign_key="team.id", primary_key=True)
    hero_id: int | None = Field(default=None, foreign_key="hero.id", primary_key=True)


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="teams", link_model=HeroTeamLink)


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    teams: list[Team] = Relationship(back_populates="heroes", link_model=HeroTeamLink)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond",
            secret_name="Dive Wilson",
            teams=[team_z_force, team_preventers],
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            teams=[team_preventers],
        )
        hero_spider_boy = Hero(
            name="Spider-Boy", secret_name="Pedro Parqueador", teams=[team_preventers]
        )
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Deadpond:", hero_deadpond)
        print("Deadpond teams:", hero_deadpond.teams)
        print("Rusty-Man:", hero_rusty_man)
        print("Rusty-Man Teams:", hero_rusty_man.teams)
        print("Spider-Boy:", hero_spider_boy)
        print("Spider-Boy Teams:", hero_spider_boy.teams)
```

```python
def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

## Create Heroes

As we have done before, we'll create a function `create_heroes()` and we'll create some teams and heroes in it:

**Python 3.10+**

```python
# Code above omitted 👆

def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond",
            secret_name="Dive Wilson",
            teams=[team_z_force, team_preventers],
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            teams=[team_preventers],
        )
        hero_spider_boy = Hero(
            name="Spider-Boy", secret_name="Pedro Parqueador", teams=[team_preventers]
        )

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine


class HeroTeamLink(SQLModel, table=True):
    team_id: int | None = Field(default=None, foreign_key="team.id", primary_key=True)
    hero_id: int | None = Field(default=None, foreign_key="hero.id", primary_key=True)


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="teams", link_model=HeroTeamLink)


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    teams: list[Team] = Relationship(back_populates="heroes", link_model=HeroTeamLink)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond",
            secret_name="Dive Wilson",
            teams=[team_z_force, team_preventers],
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            teams=[team_preventers],
        )
        hero_spider_boy = Hero(
            name="Spider-Boy", secret_name="Pedro Parqueador", teams=[team_preventers]
        )
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Deadpond:", hero_deadpond)
        print("Deadpond teams:", hero_deadpond.teams)
        print("Rusty-Man:", hero_rusty_man)
        print("Rusty-Man Teams:", hero_rusty_man.teams)
        print("Spider-Boy:", hero_spider_boy)
        print("Spider-Boy Teams:", hero_spider_boy.teams)
```

```python
def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

This is very similar to what we have done before.

We create a couple of teams, and then three heroes.

The only new detail is that instead of using an argument `team` we now use `teams`, because that is the name of the new **relationship attribute**. And more importantly, we pass a **list of teams** (even if it contains a single team).

See how **Deadpond** now belongs to the two teams?

## Commit, Refresh, and Print

Now let's do as we have done before, `commit` the **session**, `refresh` the data, and print it:

Python 3.10+

```python
# Code above omitted 👆

def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond",
            secret_name="Dive Wilson",
            teams=[team_z_force, team_preventers],
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            teams=[team_preventers],
        )
        hero_spider_boy = Hero(
            name="Spider-Boy", secret_name="Pedro Parqueador", teams=[team_preventers]
        )
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Deadpond:", hero_deadpond)
        print("Deadpond teams:", hero_deadpond.teams)
        print("Rusty-Man:", hero_rusty_man)
        print("Rusty-Man Teams:", hero_rusty_man.teams)
        print("Spider-Boy:", hero_spider_boy)
        print("Spider-Boy Teams:", hero_spider_boy.teams)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine


class HeroTeamLink(SQLModel, table=True):
    team_id: int | None = Field(default=None, foreign_key="team.id", primary_key=True)
    hero_id: int | None = Field(default=None, foreign_key="hero.id", primary_key=True)


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="teams", link_model=HeroTeamLink)


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    teams: list[Team] = Relationship(back_populates="heroes", link_model=HeroTeamLink)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond",
            secret_name="Dive Wilson",
            teams=[team_z_force, team_preventers],
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            teams=[team_preventers],
        )
        hero_spider_boy = Hero(
            name="Spider-Boy", secret_name="Pedro Parqueador", teams=[team_preventers]
        )
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Deadpond:", hero_deadpond)
        print("Deadpond teams:", hero_deadpond.teams)
        print("Rusty-Man:", hero_rusty_man)
        print("Rusty-Man Teams:", hero_rusty_man.teams)
        print("Spider-Boy:", hero_spider_boy)
        print("Spider-Boy Teams:", hero_spider_boy.teams)
```

```python
def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

## Add to Main

As before, add the `create_heroes()` function to the `main()` function to make sure it is called when running this program from the command line:

**Python 3.10+**

```python
# Code above omitted 👆

def main():
    create_db_and_tables()
    create_heroes()

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine


class HeroTeamLink(SQLModel, table=True):
    team_id: int | None = Field(default=None, foreign_key="team.id", primary_key=True)
    hero_id: int | None = Field(default=None, foreign_key="hero.id", primary_key=True)


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="teams", link_model=HeroTeamLink)


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    teams: list[Team] = Relationship(back_populates="heroes", link_model=HeroTeamLink)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond",
            secret_name="Dive Wilson",
            teams=[team_z_force, team_preventers],
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            teams=[team_preventers],
        )
        hero_spider_boy = Hero(
            name="Spider-Boy", secret_name="Pedro Parqueador", teams=[team_preventers]
        )
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Deadpond:", hero_deadpond)
        print("Deadpond teams:", hero_deadpond.teams)
        print("Rusty-Man:", hero_rusty_man)
        print("Rusty-Man Teams:", hero_rusty_man.teams)
        print("Spider-Boy:", hero_spider_boy)
        print("Spider-Boy Teams:", hero_spider_boy.teams)
```

```python
def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

## Run the Program

If we run the program from the command line, it would output:

```bash
```

## Recap

After setting up the model link, using it with **relationship attributes** is fairly straightforward, just Python objects. ✨